

Operating System, Section 2

Assignment2 - Multi-threaded word count Document

유준혁

32212808

Department of Mobile Systems Engineering

2025-01-01

Table of Contents

1. 서론
2. 소프트웨어 요구사항 기술 (Requirements)
3. 소프트웨어 설계 (Design)
4. 소프트웨어 구현 (Implementation)
5. 시험 & 디버깅 (Verification)
6. 소프트웨어 유지보수 (Maintenance)
7. 결론

1. 서론

A. 개발 목적

본 프로젝트의 목적은 멀티 스레드 프로그래밍을 이해하고, 스레드 간 동기화 문제 (Synchronization)를 실제로 경험하고 해결하는 방법을 학습하는 것이다.

스레드는 운영체제에서 하나의 프로세스 내에서 메모리를 공유하며, 동시에 실행된다. 따라서 여러 스레드가 동일한 메모리에 접근할 때, 적절한 제어가 없을 경우 데이터 불일치나 충돌 문제가 발생할 수 있다. 본 프로젝트는 생산자-소비자 문제를 기반으로 한 프로그램을 구현함으로써, 스레드 생성과 실행, 공유 메모리 접근 제어, 조건 변수 활용 등의 실제 동기화 문제를 경험하고 해결하는 것을 목표로 한다.

또한 단순히 파일의 내용을 읽고 출력하는 수준을 넘어, 여러 개의 소비자 스레드가 파일의 내용의 문자 통계를 계산하는 기능을 추가함으로써, 스레드의 수의 변화에 따른 성능을 비교하고, 병렬 처리 최적화를 시도한다.

2. 소프트웨어 요구사항 기술 (Requirements)

A. 기능적 요구사항

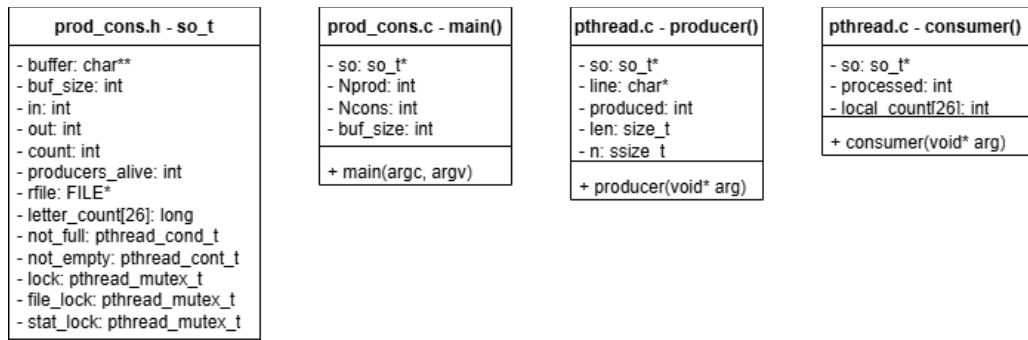
Makefile 작성	프로그램을 make 명령어로 컴파일할 수 있어야 한다.
파일 입력	프로그램을 실행하면 입력된 파일을 읽어야 한다.
스레드 동작	프로그램을 실행하면 생산자와 소비자 스레드가 입력된 수만큼 생성되어야 한다. 각 스레드는 pthread 를 이용해 생성되고 종료되어야 한다.
생산자 스레드 동작	생산자는 파일의 내용을 한 줄씩 읽고, 공유 버퍼에 저장해야 한다. 생산자는 공유 버퍼가 가득 찬 경우, 소비자가 데이터를 소비할 때까지 대기해야 한다. 생산자가 파일의 내용을 모두 읽은 경우, 소비자에게 종료 신호를 전달해야 한다.
소비자 스레드 동작	소비자는 공유 버퍼의 내용을 출력해야 한다. 소비자는 문자의 등장 횟수를 계산하고, 전역 변수에 저장해야 한다. 소비자는 버퍼가 비어 있고 생산자로부터 종료 신호를 전달받은 경우, 종료되어야 한다.
동기화 처리	생산자와 소비자의 공유 버퍼의 접근을 mutex 와 조건 변수로 제어해야 한다.
프로그램 종료	프로그램은 모든 스레드가 종료되면 문자 통계를 출력해야 한다.

B. 비기능적 요구사항

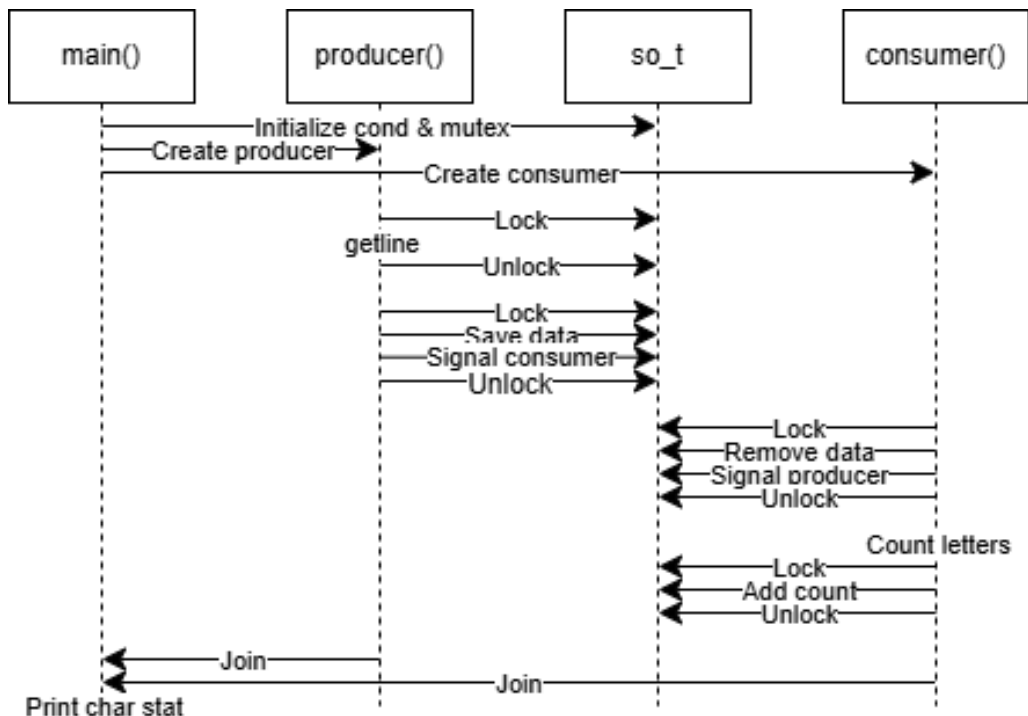
성능	프로그램의 실행 시간을 측정하고, 병렬 처리가 최적화되는 스레드의 수를 제시해야 한다.
안정성	동기화 문제가 없어야 한다. Deadlock 문제와 race condition 문제가 없어야 한다.
POSIX	POSIX 시스템 콜 사용을 준수해야 한다.
에러 처리	입력된 인자가 잘못된 경우, 오류 메시지를 출력해야 한다.

3. 소프트웨어 설계 (Design)

A. 아키텍처 설계 - 클래스 다이어그램



B. 아키텍처 설계 - 시퀀스 다이어그램



4. 소프트웨어 구현 (Implementation)

A. 개발 환경 & 도구

Operating System	Ubuntu 22.04.5 LTS
Kernel	Linux 5.15.0-136-generic
Integrated Development Environment & Editor	vi
Programming language	C
Framework	ctype.h: Character library errno.h: Error library pthread.h: POSIX thread API stdio.h: Standard input/output stdlib.h: Standard library string.h: String library time.h: Time library unistd.h: POSIX API
Build Tool	GCC compiler with Makefile

B. 모듈별 기능 설명

prod_cons.h	공유 버퍼를 정의한다. 함수 프로토타입을 선언한다.
prod_cons.c	사용자의 입력을 처리한다. 공유 버퍼를 초기화하고 동적 메모리를 할당한다. 생산자와 소비자 스레드를 처리한다. 각 알파벳의 수와 프로그램 실행 시간을 출력한다.
pthread.c	생산자와 소비자를 구현한다. Mutex 를 사용해 공유 버퍼 접근을 제어한다. 조건 변수를 사용해 생산자와 소비자를 동기화한다. 알파벳의 수를 전역 배열에 합산한다.
char_stat.c	각 알파벳의 수를 출력한다.

C. 주요 알고리즘 설명 & Code Snippet

```
// pthread.c
void *producer(void *arg){
    // 생략

    while(1){
        // Lock for multiple producer cannot read file
        simultaneously & Get line
        pthread_mutex_lock(&so->file_lock);
        n = getline(&line, &len, so->rfile);
        pthread_mutex_unlock(&so->file_lock);
        // 생략

        // Lock for producers and consumers cannot access
        shared buffer simultaneously
        pthread_mutex_lock(&so->lock);
        // Wait if shared buffer is full
        while(so->count == so->buf_size){
            pthread_cond_wait(&so->not_full, &so->lock);
        }
        // Save data into shared buffer
        so->buffer[so->in] = entry;
        so->in = (so->in + 1) % so->buf_size;
        so->count++;
        produced++;
        // Signal consumers & unlock
        pthread_cond_signal(&so->not_empty);
        pthread_mutex_unlock(&so->lock);
    }

    // 생략

    // Lock for producers and consumers cannot access shared
    variables simultaneously
    pthread_mutex_lock(&so->lock);
    so->producers_alive--;
    // Wake up all consumers so they can re-check buffer state
    or detect that producers have finished
    pthread_cond_broadcast(&so->not_empty);
    pthread_mutex_unlock(&so->lock);

    return NULL;
}
```

생산자는 다음과 같은 경우에 pthread_mutex_lock()을 사용한다.

파일을 읽을 때, 여러 생산자가 파일을 동시에 읽지 못하도록 하기 위한 경우.

정보를 공유 버퍼에 저장할 때, 생산자와 소비자가 공유 버퍼에 동시에 접근하기 못하도록 하기 위한 경우.

종료할 때, 생산자와 소비자가 공유 변수에 동시에 접근하지 못하도록 하기 위한 경우.

```

void *consumer(void *arg){
    // 생략

    while(1){
        // Lock for producers and consumers cannot access
        shared buffer simultaneously
        pthread_mutex_lock(&so->lock);
        while(so->count == 0 && so->producers_alive > 0){
            // Wait if shared buffer is empty, but producer is
            running
            pthread_cond_wait(&so->not_empty, &so->lock);
        }
        // Exit if shared buffer is empty and producers are
        not alive
        if(so->count == 0 && so->producers_alive == 0){
            pthread_mutex_unlock(&so->lock);
            break;
        }

        // Remove data from shared buffer
        char *entry = so->buffer[so->out];
        so->buffer[so->out] = NULL;
        so->out = (so->out + 1) % so->buf_size;
        so->count--;
        // Signal producer & unlock
        pthread_cond_signal(&so->not_full);
        pthread_mutex_unlock(&so->lock);

        // 생략
    }

    // Lock for multiple consumer cannot add counted letters
    to global array simultaneously
    pthread_mutex_lock(&so->stat_lock);
    for(int i = 0; i < 26; ++i){
        so->letter_count[i] += local_counts[i];
    }
    pthread_mutex_unlock(&so->stat_lock);

    return NULL;
}

```

소비자는 다음과 같은 경우에 pthread_mutex_lock()을 사용한다.

파일을 읽을 때, 생산자와 소비자가 공유 버퍼에 동시에 접근하지 못하도록 하기 위한 경우.

정보를 공유 배열에 저장할 때 여러 개의 소비자가 동시에 접근하지 못하도록 하기 위한 경우.

5. 시험 & 디버깅 (Verification)

시험 목록	입력	출력	상태
Makefile 빌드	make	.o 파일 생성	Pass
Makefile 클리어	make clear	.o 파일 삭제	Pass
인자 없음	./word_count	Usage: ./prod_cons [readfile] [producer] [consumer] [buf_size]	Pass
readfile 인자	./ word_count READ.md	READ.md, 각 알파벳의 수 출력	Pass
producer 인자	./ word_count Read.md 2	READ.md, 각 알파벳의 수 출력	Pass
consumer 인자	./ word_count Read.md 2 4	READ.md, 각 알파벳의 수 출력	Pass
buf_size 인자	./ word_count Read.md 2 4 8	READ.md, 각 알파벳의 수 출력	Pass
잘못된 인자	./ word_count hello.txt	fopen: No such file or directory	Pass

아래와 같이 스레드 수와 공유 버퍼의 크기에 따른 프로그램의 실행 시간을 측정하는 시험을 하였다.

공유 버퍼의 크기: 2			
생산자의 수: 2	소비자의 수: 2	결과: 0.040764	순위: 16
	소비자의 수: 4	결과: 0.042667	순위: 20
	소비자의 수: 8	결과: 0.051770	순위: 26
생산자의 수: 4	소비자의 수: 2	결과: 0.040866	순위: 17
	소비자의 수: 4	결과: 0.041967	순위: 19
	소비자의 수: 8	결과: 0.052209	순위: 27
생산자의 수: 8	소비자의 수: 2	결과: 0.051613	순위: 25
	소비자의 수: 4	결과: 0.049358	순위: 24
	소비자의 수: 8	결과: 0.047103	순위: 23
공유 버퍼의 크기: 4			
생산자의 수: 2	소비자의 수: 2	결과: 0.026378	순위: 6
	소비자의 수: 4	결과: 0.031266	순위: 11
	소비자의 수: 8	결과: 0.044223	순위: 22
생산자의 수: 4	소비자의 수: 2	결과: 0.028346	순위: 8
	소비자의 수: 4	결과: 0.029091	순위: 9
	소비자의 수: 8	결과: 0.037093	순위: 13
생산자의 수: 8	소비자의 수: 2	결과: 0.043016	순위: 21
	소비자의 수: 4	결과: 0.039143	순위: 15
	소비자의 수: 8	결과: 0.031159	순위: 10
공유 버퍼의 크기: 8			
생산자의 수: 2	소비자의 수: 2	결과: 0.019404	순위: 1
	소비자의 수: 4	결과: 0.025870	순위: 5
	소비자의 수: 8	결과: 0.041755	순위: 18
생산자의 수: 4	소비자의 수: 2	결과: 0.022343	순위: 3
	소비자의 수: 4	결과: 0.021279	순위: 2
	소비자의 수: 8	결과: 0.031456	순위: 12
생산자의 수: 8	소비자의 수: 2	결과: 0.037736	순위: 14
	소비자의 수: 4	결과: 0.028028	순위: 7
	소비자의 수: 8	결과: 0.024732	순위: 4

위와 같이 생산자 수가 2, 소비자 수가 2 일 때 프로그램의 실행 속도가 가장 빠르며, 공유 버퍼의 크기는 적당히 큰 경우 성능이 향상됨을 확인할 수 있다.

6. 소프트웨어 유지보수 (Maintenance)

모듈화 설계 & 확장성	기능을 각각의 모듈로 분리하였다. 각 모듈이 단일 책임 원칙 (SRP, Single Responsibility)을 준수하기 때문에 코드 수정 시 다른 모듈에 대한 영향을 최소화했다. 예를 들어, char_stat.c 모듈을 수정하는 것만으로도 각 알파벳의 수 출력 형식을 바꿀 수 있다.
에러 처리	파일 입출력, 메모리 할당 실패, 스레드 생성 실패에 대한 오류 처리 코드가 포함되어 있어, 유지보수 중에도 안정성을 확보할 수 있다.
코드 가독성	모든 모듈과 함수에 대한 주석을 작성하여 코드 이해도를 높였다.

7. 결론

스레드의 수가 많으면 컨텍스트 스위칭 오버헤드가 발생하고, 공유 버퍼의 크기가 작으면 생산자가 대기하는 시간이 길어져 성능이 오히려 떨어진다는 것을 알게 되었다. 실험 결과, 생산자와 소비자가 각각 1개, 버퍼의 크기가 8일 때 가장 빠르게 동작하였으며, 이를 통해 스레드의 수와 버퍼 크기의 적절한 조합이 성능 최적화에 매우 중요함을 알게 되었다.

다만 일부 극단적인 설정(스레드의 수: 100, 버퍼의 크기: 1)에서는 안정성 테스트가 부족하였다. 운영체제가 동적 스레드 풀을 통해 스레드 수를 자동으로 조절하는 방식에 대한 이해가 있다면, 더 효율적인 코드 작성이 가능할 것이라 생각한다.

멀티 스레드 프로그래밍을 실습하며 동기화 관리의 중요성을 직접 체감할 수 있었고, 스레드 설계, 조건 변수, 뮤텍스 활용이 실무에서 필수적인 기초 지식임을 느꼈다. GitHub에서 다른 학생들의 코드를 참고하고자 했으나 해석이 어려워 활용하지 못하였고, 코드 해석 능력의 부족도 실감하였다.